

2-SAT: A Black-Box Reference

What it solves, how to phrase constraints for it, and the template.

1. When does 2-SAT apply?

Two conditions. Both must hold.

- **Every object has exactly two states.** One boolean per object. Three or more choices per object and 2-SAT does not apply.
- **Every single condition mentions at most two objects.** This is per condition, not overall. You can have 200,000 booleans; what matters is that no individual condition needs three of them at once.

The precise test: can the condition be written as an OR of at most two literals? A literal is "x is true" or "x is false".

A condition about many objects is fine if it splits into independent one-object facts. "No treasure anywhere in positions 5 to 90" looks big, but it is 86 separate statements, each about one position. That is an AND, and ANDs always split into separate clauses. What does not split is an OR over three or more: "treasure at p, or at q, or at r" is a single choice among three and cannot be broken into pairs. That is 3-SAT, and there is no polynomial algorithm for it.

Rule of thumb: ANDs are free, ORs are capped at two.

2. Translating conditions into clauses

A clause is an OR of literals that must come out true. Every constraint you feed the solver must be in that shape. The translations below are all derived the same way: restate the condition as "at least one of these two literals is true".

Condition in words	Clause	Why
At least one of u, v	(u OR v)	Already in clause form.
Not both u and v	(NOT u OR NOT v)	If they are not both true, at least one is false.
If u then v	(NOT u OR v)	Either u is false, or v is true. Same statement.
u must be true	(u OR u)	A one-literal clause. Nothing else can satisfy it.
u must be false	(NOT u OR NOT u)	Same, negated.
Exactly one of u, v	both of the above two	At least one, and not both.
u and v must agree	(NOT u OR v) and (u OR NOT v)	Each forces the other.

The one that trips people up is "not both". Read it slowly: it is not the case that u and v are both true. So at least one of them is false. "At least one is false" is the OR of the two negative literals. That is the clause.

"If u then v" is the other one worth internalising, because implications are how the solver thinks internally. An implication is false only when the front is true and the back is false, which is exactly when NOT u and v are both false. So the implication and the clause (NOT u OR v) are the same statement.

3. How to use the template

Four steps, all in your solve function. You never touch the internals.

Step 1 — name the boolean.

Decide what "true" means and write it down. "Treasure at island i." "Item i goes left." "Node i is in the set."
Ambiguity here causes every later bug.

Step 2 — construct the solver.

```
TwoSat ts(n); // n booleans, indexed 0 .. n-1
```

Step 3 — emit one call per condition.

```
ts.addClause(lit(u, true), lit(v, true)); // at least one of u, v
ts.addClause(lit(u, false), lit(v, false)); // not both u and v
ts.addClause(lit(u, false), lit(v, true)); // if u then v
ts.addUnit(lit(u, true)); // u must be true
ts.addUnit(lit(u, false)); // u must be false
```

`lit(i, true)` is the literal "boolean i is true"; `lit(i, false)` is its negation. Order inside a clause does not matter.

Step 4 — solve and read the result.

```
string res;
if (!ts.solve(res)) { /* impossible */ }
// res[i] == '1' means boolean i is true
```

Anything global goes outside.

Conditions that are not about one or two booleans cannot be expressed as clauses. "At least one true overall", "exactly k true", "maximise the count" — handle these before or after the solver, by hand. 2-SAT finds *some* satisfying assignment; it does not optimise.

4. Complexity

	Cost
Time	$O(V + C)$ where V = number of booleans, C = number of clauses
Space	$O(V + C)$
Nodes built internally	$2V$ (one per literal)
Edges built internally	$2C$ (each clause becomes two implications)

Linear in the size of what you feed it, so the real risk is **clause count**, not the algorithm. If a condition naively expands to one clause per pair over a range — "this index conflicts with everything in $[l, r]$ " — you get $O(n^2)$ clauses and time out. Compress first: use a difference array to find which indices are forced, then emit one unit clause per forced index. That is the trick that keeps the count linear.

For a typical competitive-programming limit of n up to 10^6 booleans and a comparable number of clauses, this runs comfortably.

5. The template

Copy this into your library. Tarjan is written iteratively because a recursive version overflows the stack on large inputs.

```
struct TwoSat {
    int n;
    vector<vector<int>> g;    // implication graph on 2n nodes

    TwoSat(int n) : n(n), g(2 * n) {}

    // literal ids: 2*i = "x_i is true", 2*i+1 = "x_i is false"
    static int lit(int i, bool val) { return 2 * i + (val ? 0 : 1); }

    void addClause(int a, int b) {          // (a OR b)
        g[a ^ 1].push_back(b);             // if a is false, b must hold
        g[b ^ 1].push_back(a);             // if b is false, a must hold
    }
    void addUnit(int a) { addClause(a, a); } // a must hold

    bool solve(string &res) {
        int N = 2 * n;
        vector<int> num(N, 0), low(N, 0), comp(N, -1), idx(N, 0), stk, call;
        int timer = 0, nc = 0;

        for (int s = 0; s < N; s++) {
            if (num[s]) continue;
            call.push_back(s);
            while (!call.empty()) {
                int v = call.back();
                if (!num[v]) { num[v] = low[v] = ++timer; stk.push_back(v); }
                bool down = false;
                while (idx[v] < (int)g[v].size()) {
                    int u = g[v][idx[v]++];
                    if (!num[u]) { call.push_back(u); down = true; break; }
                    else if (comp[u] == -1) low[v] = min(low[v], num[u]);
                }
                if (down) continue;
                call.pop_back();
                if (low[v] == num[v]) {
                    while (true) {
                        int u = stk.back(); stk.pop_back();
                        comp[u] = nc;
                        if (u == v) break;
                    }
                    nc++;
                }
                if (!call.empty())
                    low[call.back()] = min(low[call.back()], low[v]);
            }
        }

        res.assign(n, '0');
        for (int i = 0; i < n; i++) {
            if (comp[2 * i] == comp[2 * i + 1]) return false; // contradiction
            if (comp[2 * i] < comp[2 * i + 1]) res[i] = '1';
        }
        return true;
    }
};
```

Two lines are worth knowing even as a black-box user. The failure condition is `comp[2i] == comp[2i+1]`: a boolean and its negation forced into each other, so neither value works. And the assignment line compares component indices — the direction is Tarjan-specific, so if you ever swap in a Kosaraju template, that comparison flips.

Unconstrained booleans — ones no clause mentions — come out true with this template, since node $2i$ is reached before node $2i+1$.

6. Problem patterns that are secretly 2-SAT

Disguise	The boolean
Each item goes in one of two positions	item i takes position A
Each element is kept or removed	element i is kept
Each interval is placed left or right of a point	interval i goes left
Each edge is oriented one way or the other	edge i points forward
Each person is assigned to one of two groups	person i is in group A
Each value is flipped or left alone	value i is flipped

Once the boolean is named, the constraints usually read off directly: "these two cannot both be in group A" becomes a not-both clause, "if this one goes left then that one must too" becomes an implication.

A common trick: **at most one of three or more** is expressible — write a not-both clause for every pair. It is only **at least one of three or more** that breaks, since that needs three literals in a single OR. So if a problem gives you the at-least-one part from its structure and only asks you to encode exclusions, you are still inside 2-SAT.

7. The shortcut — check before you reach for the template

Look at your finished clause list. If **every two-literal clause is all-positive** (of the form "at least one of these is true") and negations only ever appear alone as unit clauses, you do not need the solver at all.

In that case: set every boolean true except the ones forced false, then verify. That assignment satisfies the instance whenever any assignment does, because making more booleans true can only help positive clauses and cannot disturb the forced ones.

The mirror case works the same way: all-negative two-literal clauses with positive units only, so set everything false except what is forced true.

You genuinely need the solver when some clause **mixes a positive and a negative literal**. That is where the tension lives — one clause wants a boolean true, another wants it false depending on a third, and the choice has to propagate. No mixing, no tension, no need for SCCs.

8. Pitfalls

- **Adding only one implication edge per clause.** The classic bug. It does not crash; it silently returns wrong answers. Always both directions.
- **Quadratic clause generation.** Ranges and "conflicts with everything in this window" expand badly. Compress with a difference array or prefix structure first.
- **Recursive Tarjan.** Stack overflow on large inputs. Use the iterative version.
- **Forgetting the global condition.** The solver has no idea you also needed at least one true overall. Check it yourself after.
- **Porting the assignment line between templates.** The component-index comparison direction depends on which SCC algorithm produced the numbering.